

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики и кибернетики

Кафедра технической кибернетики

Отчет по лабораторной работе №3

Дисциплина: «ООП»

Тема «Реализация табулированных функций на основе связанного
списка и обработка исключений»

Выполнил: Булавкина

Виктория Станиславовна

Группа: 6201-120303D

Самара, 2025

Задание 1

Я продолжила работу с пакетом functions. Создала новые классы исключений для обработки ошибок, возникающих при работе с табулированными функциями:

InappropriateFunctionPointException – для случаев нарушения порядка точек по оси X или попытки добавить точку с уже существующим X.

FunctionPointIndexOutOfBoundsException – для ситуаций выхода за границы индексов при обращении к точкам функции. Эти классы наследуются от стандартных исключений Java и позволяют точно идентифицировать источник ошибки.

Задание 2

Разработала иерархию классов для представления табулированных функций:

- TabulatedFunction – интерфейс, определяющий общий контракт для всех табулированных функций.
- ArrayTabulatedFunction – реализация функции на основе массива точек.
- LinkedListTabulatedFunction – реализация функции на основе двусвязного списка.

Обе реализации поддерживают одинаковый набор операций, что обеспечивает полиморфизм и возможность взаимозаменяемости.

```

package functions;

public class ArrayTabulatedFunction implements TabulatedFunction { 5 usages new *
    private FunctionPoint[] points; 33 usages
    private int size; 19 usages
    private int capacity; 8 usages

    public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) { 2 usages new *
        if (pointsCount < 2) {
            throw new IllegalArgumentException("Количество точек должно быть не менее 2");
        }
        if (leftX >= rightX) {
            throw new IllegalArgumentException("Левая граница должна быть меньше правой");
        }

        capacity = Math.max(pointsCount * 2, 10);
        points = new FunctionPoint[capacity];
        size = pointsCount;

        double step = (rightX - leftX) / (pointsCount - 1);
        for (int i = 0; i < pointsCount; i++) {
            double x = leftX + i * step;
            points[i] = new FunctionPoint(x, y: 0);
        }
    }
}

```

```

package functions;

1 public interface TabulatedFunction { 11 usages 2 implementations viktoriabulavkina7-sys *
    // Основные методы
2 double getLeftDomainBorder(); 2 usages 2 implementations new *
3 double getRightDomainBorder(); 2 usages 2 implementations new *
4 double getFunctionValue(double x); 4 usages 2 implementations new *
5 int getPointsCount(); no usages 2 implementations new *

    // Методы работы с точками
6 FunctionPoint getPoint(int index); 1 usage 2 implementations new *
7 void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException; 3 usages 2 implementations new *
8 double getPointX(int index); no usages 2 implementations new *
9 void setPointX(int index, double x) throws InappropriateFunctionPointException; no usages 2 implementations new *
10 double getPointY(int index); no usages 2 implementations new *
11 void setPointY(int index, double y); no usages 2 implementations new *
12 void deletePoint(int index); 2 usages 2 implementations new *
13 void addPoint(FunctionPoint point) throws InappropriateFunctionPointException; 3 usages 2 implementations new *

    // Метод для отладки
14 void printFunction(); 4 usages 2 implementations new *
}

```

```

27 @      public ArrayTabulatedFunction(double leftX, double rightX, double[] values) { 4
28         this(leftX, rightX, values.length);
29         for (int i = 0; i < values.length; i++) {
30             points[i].setY(values[i]);
31         }
32     }
33
34     @Override 2 usages new *
35     public double getLeftDomainBorder() {
36         return points[0].getX();
37     }
38
39     @Override 2 usages new *
40     public double getRightDomainBorder() {
41         return points[size - 1].getX();
42     }
43
44     @Override 4 usages new *
45     public double getFunctionValue(double x) {
46         if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
47             return Double.NaN;
48         }
49
50         int i = 0;
51         while (i < size - 1 && x > points[i + 1].getX()) {
52             i++;
53         }
54
55         if (Math.abs(x - points[i].getX()) < 1e-10) {
56             return points[i].getY();
57         }
58         if (i < size - 1 && Math.abs(x - points[i + 1].getX()) < 1e-10) {
59             return points[i + 1].getY();
60         }

```

```

1 package functions;
2
3 public class LinkedListTabulatedFunction implements TabulatedFunction { 4 usages new *
4
5     // Внутренний класс для элемента списка
6     private static class FunctionNode { 23 usages new *
7         FunctionPoint point; 27 usages
8         FunctionNode prev; 18 usages
9         FunctionNode next; 30 usages
10
11         FunctionNode(FunctionPoint point) { 3 usages new *
12             this.point = point;
13             this.prev = null;
14             this.next = null;
15         }
16     }
17
18     private FunctionNode head; // выделенная голова (не хранит данные) 28 usages
19     private FunctionNode lastAccessedNode; 16 usages
20     private int lastAccessedIndex; 14 usages
21     private int size; 15 usages
22
23     // Конструкторы
24     public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) { 1 usa
25         if (pointsCount < 2) {
26             throw new IllegalArgumentException("Количество точек должно быть не менее 2");
27         }
28         if (leftX >= rightX) {
29             throw new IllegalArgumentException("Левая граница должна быть меньше правой");
30         }

```

Задание 3

В классе ArrayTabulatedFunction реализовала методы:

- Конструкторы с проверкой входных данных (границы, количество точек).
- Методы доступа к точкам с валидацией индексов.
- Метод getFunctionValue() с линейной интерполяцией.
- Методы addPoint(), deletePoint(), setPoint() с обработкой исключений.
- Автоматическое расширение массива при добавлении точек

```
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) { 2 usages ne
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2");
    }
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница должна быть меньше правой");
    }

    capacity = Math.max(pointsCount * 2, 10);
    points = new FunctionPoint[capacity];
    size = pointsCount;
}
```

```
private void checkIndex(int index) { 2 usages new*
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException(
            "Индекс " + index + " вне диапазона [0, " + (size - 1) + "]");
    }
}
```

```

public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    FunctionNode node = getNodeByIndex(index);
    FunctionPoint temp = new FunctionPoint(node.point);
    temp.setX(x);
    setPoint(index, temp);
}

@Override no usages new *
public double getPointY(int index) {
    return getNodeByIndex(index).point.getY();
}

@Override no usages new *
public void setPointY(int index, double y) {
    getNodeByIndex(index).point.setY(y);
}

@Override 2 usages new *
public void deletePoint(int index) {
    deleteNodeByIndex(index);
}

@Override 3 usages new *
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    // Находим позицию для вставки
    int insertIndex = 0;
    FunctionNode current = head.next;
    while (current != head && current.point.getX() < point.getX()) {
        current = current.next;
        insertIndex++;
    }
}

```

Задание 5

Создала класс Main для тестирования обеих реализаций. В нём реализованы:

- Метод testFunction() – тестирование основных операций для переданной функции.
- Метод testExceptions() – проверка обработки исключений в различных сценариях.
- Демонстрация работы интерполяции, добавления, удаления и изменения точек.
- Сравнение работы ArrayTabulatedFunction и LinkedListTabulatedFunction.

Задание 6

В ходе тестирования проверяла следующие сценарии:

- Корректность интерполяции внутри и вне области определения.
- Обработка некорректных входных данных в конструкторах.
- Проверка граничных значений индексов.
- Попытки нарушения порядка точек по X.
- Удаление точек при минимальном количестве.
- Добавление дублирующихся точек. Все исключения корректно обрабатываются, программа не завершается аварийно.

Ниже прикрепила скриншоты консольного вывода запуска main.

=== ТЕСТИРОВАНИЕ ARRAY TABULATED FUNCTION ===

Табулированная функция (массив):

```
( 0,00; 0,00)
( 1,00; 1,00)
( 2,00; 4,00)
( 3,00; 9,00)
( 4,00; 16,00)
( 5,00; 25,00)
```

Значение в точке 2.5: 6.5

После добавления точки (2.5, 6.25):

Табулированная функция (массив):

```
( 0,00; 0,00)
( 1,00; 1,00)
( 2,00; 4,00)
( 2,50; 6,25)
( 3,00; 9,00)
( 4,00; 16,00)
( 5,00; 25,00)
```

=== ТЕСТИРОВАНИЕ LINKED LIST TABULATED FUNCTION ===

Табулированная функция (список):

```
( 0,00; 0,00)
( 1,00; 1,00)
( 2,00; 4,00)
```

Значение в точке 2.5: 6.5

После добавления точки (2.5, 6.25):

Табулированная функция (список):

(0,00; 0,00)

(1,00; 1,00)

(2,00; 4,00)

(2,50; 6,25)

(3,00; 9,00)

(4,00; 16,00)

(5,00; 25,00)

=== ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ ===

1. Тест некорректного конструктора:

Поймано: `IllegalArgumentException` - Левая граница должна быть меньше правой

2. Тест выхода за границы массива:

Поймано: `FunctionPointIndexOutOfBoundsException` - Индекс 10 вне диапазона [0, 2]

3. Тест некорректной точки:

Поймано: `InappropriateFunctionPointException` - X новой точки должен быть меньше X следующей

4. Тест удаления при минимальном количестве точек:

Поймано: `IllegalStateException` - Нельзя удалить точку: минимум 2 точки

5. Тест дублирования точки:

Поймано: `InappropriateFunctionPointException` - Точка с таким X уже существует

6. Тест совместимости интерфейса:

Array функция работает: 1.0

LinkedList функция работает: 1.0

Обе функции реализуют один интерфейс!